4

演習 4　セッションと入力チェック

演習の目標

■　ID とパスワードによる認証処理を実装できる
■　セッションへのデータ保持と、活用方法について理解できる
■　入力チェックとエラーハンドリングを理解し、実装できる

演習の概要

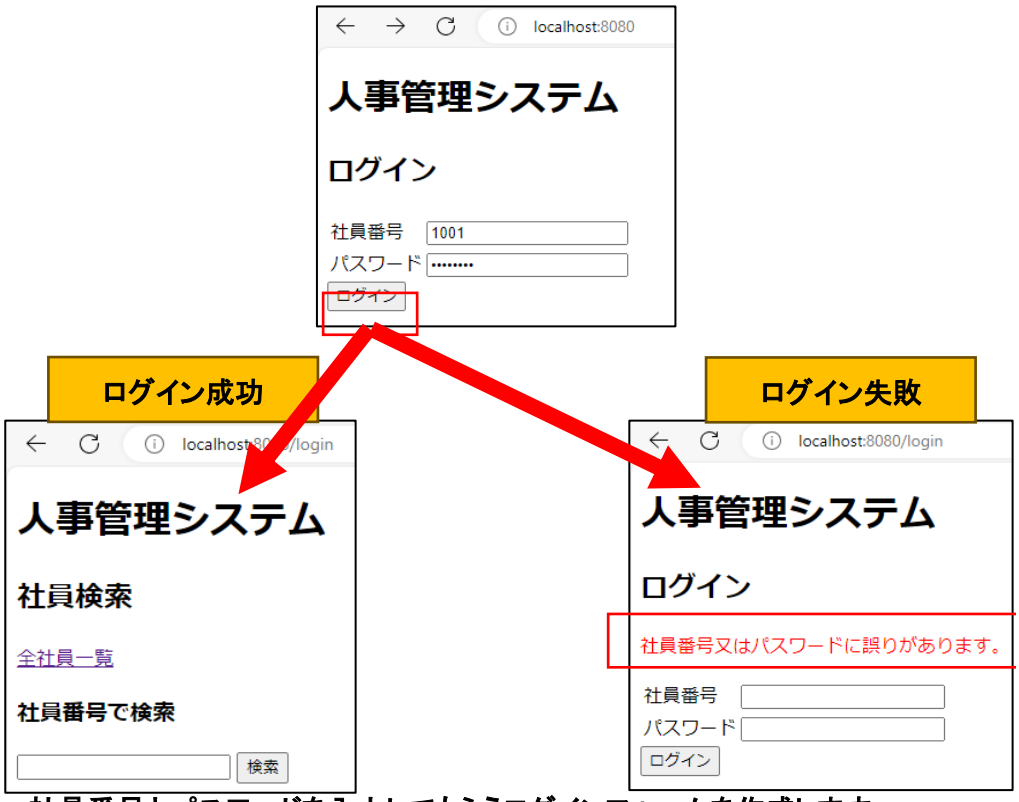
第4 章で学習したセッションと入力チェックの基本文法を確認し、実際にデータベースと連動するログインプログラムを作成します。

■　ID とパスワードの整合性をチェックする機能を持つメソッドの作成■　ログイン成功時、ユーザ情報をセッションに格納する機能を持つメソッドの作成■　入力値の整合性をチェックする機能を持つメソッドと画面の作成

予想所要時間

演習4.1 10 分
演習4.2 20 分
演習4.3 20 分

演習4.1. ID とパスワードからのログイン機能の実装



■ **社員番号とパスワードを入力してもらうログインフォームを作成します。**

- 入力値に誤りがない場合は検索画面へ遷移
- 入力値に誤りがある場合は、ログイン失敗画面へ遷移し、再入力を促します。

a. src/main/resources/templates/index.html のソースコードを確認してください。

- テキスト4.1 の【作業手順②】の内容が該当します。
- name 属性の値を覚えてください。
- form タグのaction 属性値を確認してください。

b. EmpRepository に、id とpassword のAND 検索を行う抽象メソッドを追加してください。

- テキスト4.1 の【作業手順③】の内容を参考にしてください。
- 主キーとのAND 検索ですので、必ず1 行のデータを取得します。よって、戻り値型はエンティティのEmployee です。
- この演習では、「empId」ではなく、「Id」です。間違えるとエラーが出ます。注意

c. LoginController のlogin メソッドの引数を入れてください。

- テキスト4.1 の【作業手順④】を参考にしてください。
- a.で送信したデータを受け取るメソッドです。(POST で送信しているので、@PostMapping です)
- login メソッドの引数は、id と password の変数で受け取っても良いですし、empFormを使って受け取っても大丈夫です。

d. ID とパスワードのAND 検索を行い、ユーザデータを取得してください。

- b.で追加したメソッドを呼び出します。
- 引数には、入力された id と password を入れてください。

e. 取得したデータがnull なら、“login_fail.html”へ遷移させる条件文を書いてください。

- redirect を使わず、普通に login_fail.html へ return するようにしてください。
- null 以外なら、正しい入力ということなので、演習 2 で実装した「検索画面」の“emp/index.html”へ遷移させます。

f. src/main/resources/templates/login_fail.html のソースコードを確認してください。

- 既存のログインページ(index.html)の<form>要素と同じコードが書かれていることを確認してください。action 属性が同じですので、同じメソッドを呼び出します。
- 「社員番号又はパスワードに誤りがあります。」が記入されていることを確認してください。

■ **動作確認：**http://localhost:8080/

● **＜正しいアカウント情報入力＞**

社員番号：1001パスワード：password を入力し、[ログイン]ボタンをクリックしてください。

→ 正常に検索画面へ遷移できたら、OK です。

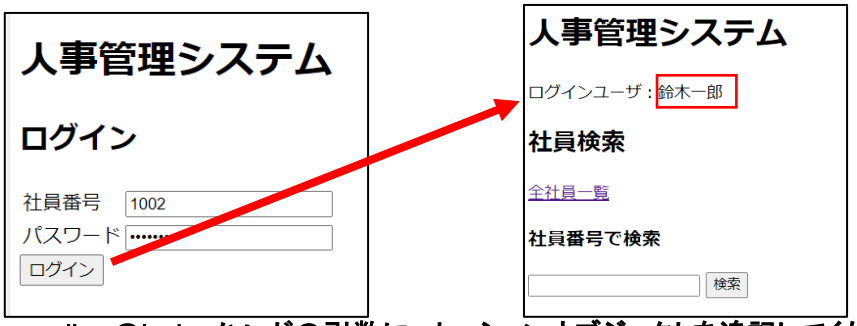
● **＜正しくないアカウント情報入力＞**

社員番号：1001パスワード：pass を入力し、[ログイン]ボタンをクリックしてください。→ ログインに失敗し、再入力を促すメッセージが表示されたら、OK です。

演習4.2. セッションデータの取得と利用

■　正しい ID とパスワードが入力されたことが確認できたら、システム全域で使いまわしできるように、ログインユーザのデータをセッションに格納しておきましょう。

■　**演習4.1 のソースコードに、追記してください。**



a. LoginController のlogin メソッドの引数に、セッションオブジェクトを追記してください。

●　テキスト4.2 の【作業手順①】を参考にしてください。

b. id とパスワードのAnd 検索から取得したユーザデータをセッションに格納してください。

●　正しいID とパスワードを入力したら、実行される箇所に追記してください。

●　セッションに格納する際のキー名は、「“user”」にしてください。

c. /resources/templates/emp/index.html にログインユーザ名を表示してください。

●　テキスト4.2 の【作業手順②】を参考にしてください。

●　</hr>の前のにth:text 属性を入れてください。

●　セッションに格納していた、“user”を使ってください。

●　ログイン画面のindex.html ではなく、/emp/index.html ですので、注意しましょう。

d. c.と同じコードを、emp フォルダ内の全てのhtml ファイルに追記してください。

■　**動作確認：http://localhost:8080/**

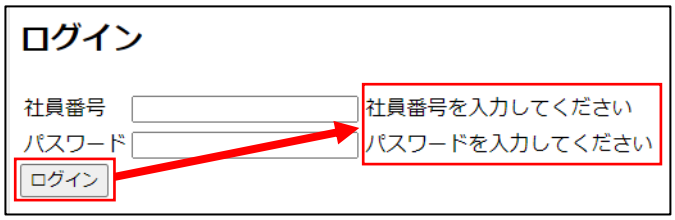
●　社員番号：1001パスワード：password　を入力し、[ログイン]ボタンをクリックしてください。

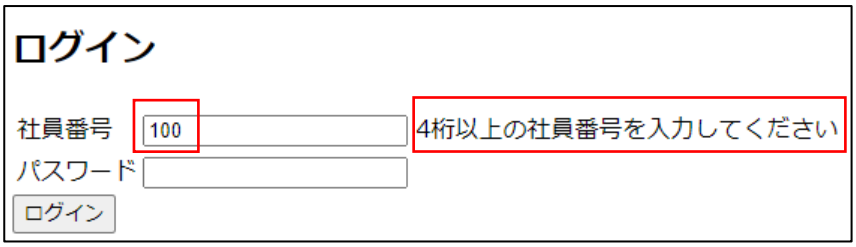
●　社員検索画面に、ログインユーザ名が表示されるかを確認してください。

●　画面を遷移しても、ユーザ名が表示されていることを確認してください。

演習4.3. 入力チェック機能の実装

■　演習 4.1 では、入力に誤りがある際には、予め用意した login_fail.html へ遷移させる方法を使いました。今度は、index.html 内に入力チェック機能を実装することで、より柔軟で保守性を高めるプログラミングに挑戦しましょう。





a. EmpForm に、以下のような入力チェックができるように、アノテーションを追記してください。

対象	チェック内容	出力メッセージ
社員番号	①未入力時 ②4 桁未満の社員番号入力時	社員番号を入力してください 4 桁以上の社員番号を入力してください
パスワード	①未入力時	パスワードを入力してください

● テキスト 4.3 の【作業手順①】を参考にしてください。● 「未満」については、「4.3.1 その他の validation アノテーション」の@Min を参考にしてください。(文字ではなく、数字の文字数チェックはなく、数値の大きさで判定)

● **@Min(value = 1000, message = “”)**のように使います。

● 複数のオプションは、,(カンマ)で並べることができます。

b. LoginController のshowLoginView メソッドの引数に、Form クラスを入れてください。

● テキスト 4.3 の【作業手順②】を参考にしてください。

● この手順を書き忘れると、エラーが出ます。

c. LoginController のlogin メソッドの引数の部分を、入力チェックができるように修正してください。

- テキスト 4.3 の【作業手順③】を参考にしてください。
- セッション関連は、そのまま残してください。

d. 入力にエラーがある場合は、ログイン画面へ戻すように条件文を入れてください。

- And 検索を行うコードの前に、記入してください。
- 入力エラーがある場合は、“index”へreturn してください。
- And 検索を行う時のlogin_fail.html へ遷移する条件文は、残しても大丈夫です。

e. /resources/templates/index.html のコードを修正してください。

- テキスト 4.3 の【作業手順④】を参考にしてください。
- form の属性に、th:object を追記してください。login メソッドのform の変数名と一致させる必要があります。
- テキストフォームのname 属性の代わりに、th:field 属性に修正してください。
- エラーがある場合、メッセージを出力できるように、<td></td>の間にを入れて、th:errors 属性を付与してください。

■ **動作確認：http://localhost:8080/**

- **＜正しいアカウント情報入力＞**

社員番号：1001パスワード：password を入力し、[ログイン]ボタンをクリックしてください。→ 正常に検索画面へ遷移でき、ユーザ名が表示されたら、OK です。

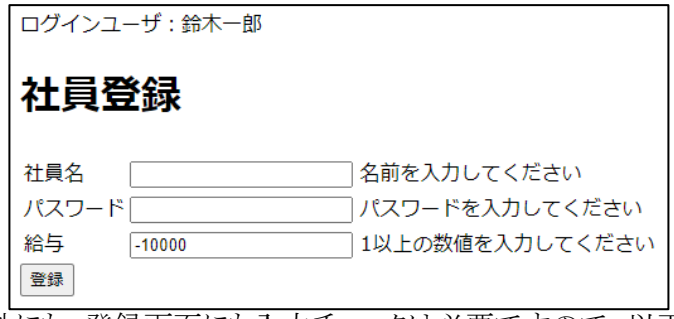
- **＜何も入力しない＞**

社員番号：【空欄】パスワード：【空欄】 の状態で [ログイン]ボタンをクリックしてください。→ ログインに失敗し、再入力を促すメッセージが表示されたら、OK です。

- **<社員番号を4 桁未満入力>**

社員番号：100パスワード：任意 の状態で [ログイン]ボタンをクリックしてください。→ ログインに失敗し、桁数についてのメッセージが表示されたら、OK です。

演習4.4.【オプション】登録画面に入力チェック機能の実装



ログイン画面以外にも、登録画面にも入力チェックは必要ですので、以下のような仕様で入力チェック機能を実装してください。社員番号のアノテーションは削除してください。

対象	チェック内容	出力メッセージ
社員番号	①未入力時	社員番号を入力してください
(アノテーシ	②4 桁未満の社員番号入力時	4 桁以上の社員番号を入力してください
ョン削除)		
パスワード	①未入力時	パスワードを入力してください
社員名	①未入力時	名前を入力してください
給与	①未入力時 ②入力値が1 未満の場合	給与を入力してください 1 以上の数値を入力してください

a. 既存のempForm をコピーし、EmpForm2.java をform パッケージ内に作成します。

- 社員番号に付与していたアノテーションは、削除してください。

b. EmpController のshowInputView メソッドを修正してください。

- 引数に入れるForm の名前は、a で追加したForm にしてください。

c. EmpController のinsertEmp メソッドを修正してください。

- 引数に入れるForm の名前は、a で追加したForm にしてください。

d. emp/emp_input.html を修正してください。

th:object に指定するForm の変数名は、c で指定したform の変数名に一致させてください。